240701593
VINOTH.V

Experiment 3

Aim

To study and understand the differences between **Command Line Interface (CLI)**, **Graphical User Interface (GUI)**, and **Voice User Interface (VUI)** and analyze their features, advantages, and limitations.

Introduction

User Interfaces allow humans to interact with computer systems. Different types of interfaces provide different ways for users to communicate with computers. The three common types are **CLI, GUI, and VUI**.

- **CLI** uses text-based commands.
- **GUI** uses graphical elements such as icons and buttons.
- **VUI** allows users to interact using voice commands.

Each interface has its own advantages and use cases depending on the user requirements.

1. Command Line Interface (CLI)

A **Command Line Interface (CLI)** is a text-based interface where users interact with the system by typing commands.

Features

- Users type commands using a keyboard.
- Requires knowledge of specific command syntax.
- Allows direct communication with the system.

Advantages

- Fast and powerful for experienced users.
- Supports automation through scripting.
- Uses very low system resources.

Disadvantages

- Difficult for beginners to learn.
- Commands must be remembered accurately.

Example

- Command Prompt in **Microsoft Windows**
- Terminal in **Linux**

Implementation Example

```
import os
import sys

print("Program started")

def rename_file(old_name, new_name):
    print(f"Trying to rename {old_name} -> {new_name}")
    try:
        os.rename(old_name, new_name)
        print(" File renamed successfully")
    except FileNotFoundError:
        print(" File not found")
    except Exception as e:
        print(" Error:", e)
```

```python
if __name___== "__main__":
    print("Arguments received:", sys.argv)

    if len(sys.argv) != 3:
        print("Usage: python rename_file_cli.py old.txt new.txt")
    else:
        rename_file(sys.argv[1], sys.argv[2])
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

(venv) PS D:\Exp3_FileRenamer> python -u "d:\Exp3_FileRenamer\rename_file_cli.py"
Program started
Arguments received: ['d:\\Exp3_FileRenamer\\rename_file_cli.py']
Usage: python rename_file_cli.py old.txt new.txt
(venv) PS D:\Exp3_FileRenamer> python rename_file_cli.py newtextfile.txt newfile.txt
Program started
Arguments received: ['rename_file_cli.py', 'newtextfile.txt', 'newfile.txt']
Trying to rename newtextfile.txt -> newfile.txt
 File renamed successfully
(venv) PS D:\Exp3_FileRenamer>
```

Example command: dir
This command displays the list of files and folders in the current directory.

2. Graphical User Interface (GUI)
A **Graphical User Interface (GUI)** allows users to interact with the system through graphical elements such as windows, icons, buttons, and menus.

Features

- Uses visual components for interaction.
- Users can click, drag, and select items.
- Easier to learn compared to CLI.

Advantages

- User-friendly and easy to understand.
- Does not require memorizing commands.
- Suitable for beginners.

Disadvantages

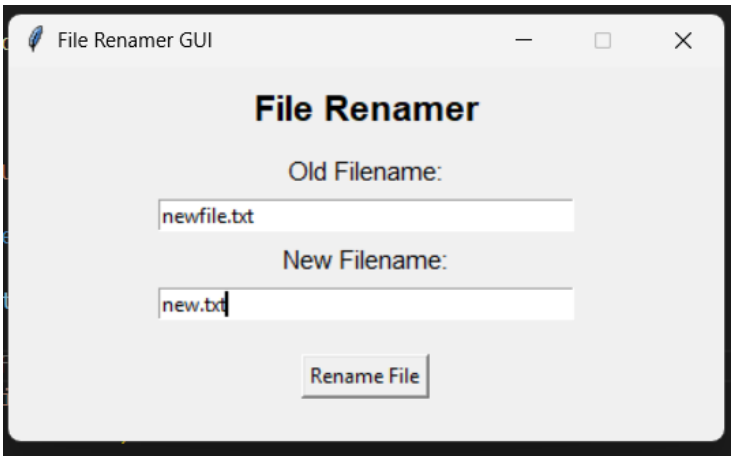
- Uses more system resources (CPU, RAM, GPU).
- Less efficient for repetitive tasks compared to CLI.

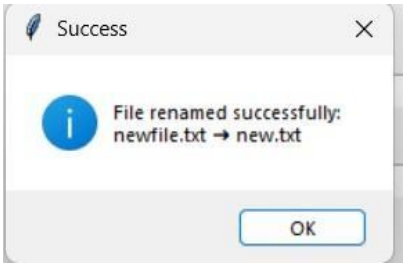
Examples

- **Microsoft Windows**
- **Android**
- **iOS**

Implementation:

```
import tkinter as tk
from tkinter import messagebox
import os

def rename_file():
    old_name = entry_old.get().strip()
    new_name = entry_new.get().strip()

    # validation
    if old_name == "" or new_name == "":
        messagebox.showwarning("Input Error", "Please enter both filenames")
        return

    try:
        os.rename(old_name, new_name)
        messagebox.showinfo("Success", f"File renamed successfully:\n{old_name} → {new_name}")
    except FileNotFoundError:
        messagebox.showerror("Error", f"File '{old_name}' not found")
    except FileExistsError:
        messagebox.showerror("Error", f"File '{new_name}' already exists")
    except Exception as e:
        messagebox.showerror("Error", str(e))

root = tk.Tk()
root.title("File Renamer GUI")
root.geometry("420x220")
root.resizable(False, False)

title = tk.Label(root, text="File Renamer", font=("Arial", 16, "bold"))
title.pack(pady=10)

tk.Label(root, text="Old Filename:", font=("Arial", 11)).pack()
entry_old = tk.Entry(root, width=40)
entry_old.pack(pady=5)

tk.Label(root, text="New Filename:", font=("Arial", 11)).pack()
entry_new = tk.Entry(root, width=40)
entry_new.pack(pady=5)

rename_btn = tk.Button(root, text="Rename File", command=rename_file)
rename_btn.pack(pady=15)

root.mainloop()
```





Users interact using:

- Mouse clicks
- Icons
- Application windows
- Menu options

3. Voice User Interface (VUI)

A **Voice User Interface (VUI)** allows users to interact with a system using voice commands.

Features

- Users communicate through speech.
- No need for keyboard or mouse.
- Useful for hands-free interaction.

Advantages

- Convenient and natural interaction.
- Faster for simple commands.
- Useful for accessibility.

Disadvantages

- Speech recognition errors may occur.
- Background noise can affect accuracy.
- Accents may sometimes cause misinterpretation.

Examples

- **Amazon Alexa**
- **Google Assistant**
- **Apple Siri**

Implementation:

```
import speech_recognition as sr
import os

def rename_file_from_voice(command):
    command = command.lower().strip()

    if not command.startswith("rename") or " to " not in command:
        print("Invalid command format")
        print("Say: rename old file.txt to new file.txt")
        return

    try:
        parts = command.replace("rename", "", 1).strip().split(" to ", 1)

        old_name = parts[0].strip()
        new_name = parts[1].strip()

        old_name = old_name.replace("file ", "")
        new_name = new_name.replace("file ", "")

        if not os.path.exists(old_name):
            print(f"File '{old_name}' not found in this folder.")

            print("\n Available files:")
            for f in os.listdir():
                print(" -", f)
            return


        confirm = input(f"⚠ Rename '{old_name}' → '{new_name}' ? (y/n): ").lower()

        if confirm != "y":
            print(" Rename cancelled")
            return

        os.rename(old_name, new_name)
        print(f" File renamed successfully: {old_name} → {new_name}")

    except Exception as e:
        print(" Error while renaming:", e)

def listen_command():
    recognizer = sr.Recognizer()

    try:
        with sr.Microphone() as source:
```

```
        print("\n Speak now...")
        print(" Example: rename test file dot txt to new file dot txt\n")

        recognizer.adjust_for_ambient_noise(source, duration=1)
        audio = recognizer.listen(source)

    try:
        command = recognizer.recognize_google(audio)
        print("🗣 You said:", command)


        command = command.replace(" dot ", ".")
        command = command.replace(" underscore ", "_")

        rename_file_from_voice(command)

    except sr.UnknownValueError:
        print("Could not understand audio")

    except sr.RequestError as e:
        print(" Network error:", e)

  except Exception as e:
    print("Microphone error:", e)

if __name___== "__main__":
  listen_command()
```

```
(venv) PS D:\Exp3_FileRenamer> python -u "d:\Exp3_FileRenamer\rename_file_vui.py"

  Speak now...
  Example: rename test file dot txt to new file dot txt

```

Examples commands:
Users give commands such as:
"Play music"
"What is the weather today?"

Result
The experiment helped in understanding the differences between **CLI, GUI, and VUI interfaces**. Each interface provides a different way of interacting with computers and is suitable for different types of users and applications.